

NAN

Private Network Service Platform

Computer Networks — Phase 1 Report

Role	Team Member
DNS Server (Mac 1)	Lakshya
Nginx Gateway (Mac 2)	Kushal
Backend A (Mac 3)	Chinmay
Backend B (Mac 4)	Prachee

Course: Computer Networks

Team: NAN

Phase: 1 — Build & Observe

Date: October 2026

“The application stays simple — the network is the project.”

A client types a private domain name. It resolves through a team-owned DNS server, connects over HTTPS through a reverse proxy, and receives a response from one of two backend servers. Every packet is captured and explained.

Contents

1	Project Overview	3
1.1	Objective	3
1.2	Team and Machine Roles	3
1.3	DNS Records	3
2	Network Architecture	3
2.1	Topology	3
2.2	Cloud Architecture Equivalents	3
3	Task A — LAN Setup	3
3.1	Overview	4
3.2	Evidence — Direct Backend Access	4
4	Task B — Private DNS Server	5
4.1	Configuration	5
4.2	Evidence — DNS Resolution	5
4.3	DNS vs. IP Layer Distinction	6
5	Task C — Backend Services	6
5.1	Design	6
5.2	Source Code — Backend A	6
6	Task D — Nginx Reverse Proxy and Load Balancer	7
6.1	Configuration	7
6.2	Traffic Path	8
6.3	Evidence — Load Balancing	8
7	Task E — HTTPS and TLS	8
7.1	Certificate Authority Setup	9
7.2	Server Certificate	9
7.3	TLS Termination Architecture	9
7.4	TLS Handshake Observed	9
8	Task F — HTTP Caching	10
8.1	Implementation	10
8.2	Evidence	10
8.3	Cache Behaviour Explained	10
9	Task G — Complete Protocol Flow Capture	11
9.1	OSI Layer Mapping	11
9.2	Port Identification	11
9.3	Wireshark Captures	12
9.4	TCP Three-Way Handshake	12
10	Phase 1 Failure Demonstrations	12

10.1 Backend Failover — Detailed	12
10.2 502 Bad Gateway	13
11 Security Notes	13
12 Project File Structure	13
13 Summary	14

1 Project Overview

1.1 Objective

The NAN project builds a fully local, four-machine distributed service platform running on a private Wi-Fi LAN — no cloud, no pre-configured servers. The goal is to demonstrate, measure, and explain every layer of a real network request: from DNS resolution, through TCP and TLS handshake, to HTTP load balancing and application response.

1.2 Team and Machine Roles

Each physical machine takes a dedicated network role. The table below maps each team member to their machine’s responsibility and the software they operate.

Table 1: Team Machine Roles and IP Addresses

Machine	Role	Member	IP Address	Port(s)	Software
Mac 1	DNS Server	Lakshya	10.7.3.114	53	dnsmasq
Mac 2	Nginx Gateway	Kushal	10.7.29.219	443, 8080	nginx + TLS
Mac 3	Backend A	Chinmay	10.7.20.70	3001	Python HTTP
Mac 4	Backend B	Prachee	10.7.0.210	3002	Python HTTP

1.3 DNS Records

Two private domain names were configured, both resolving to the Nginx gateway:

app.NAN.test	->	10.7.29.219	(Nginx Gateway)
api.NAN.test	->	10.7.29.219	(Nginx Gateway)

2 Network Architecture

2.1 Topology

All four machines connect to the same private Wi-Fi access point. There is no cloud component — the entire network runs locally on the team’s laptops.

2.2 Cloud Architecture Equivalents

3 Task A — LAN Setup

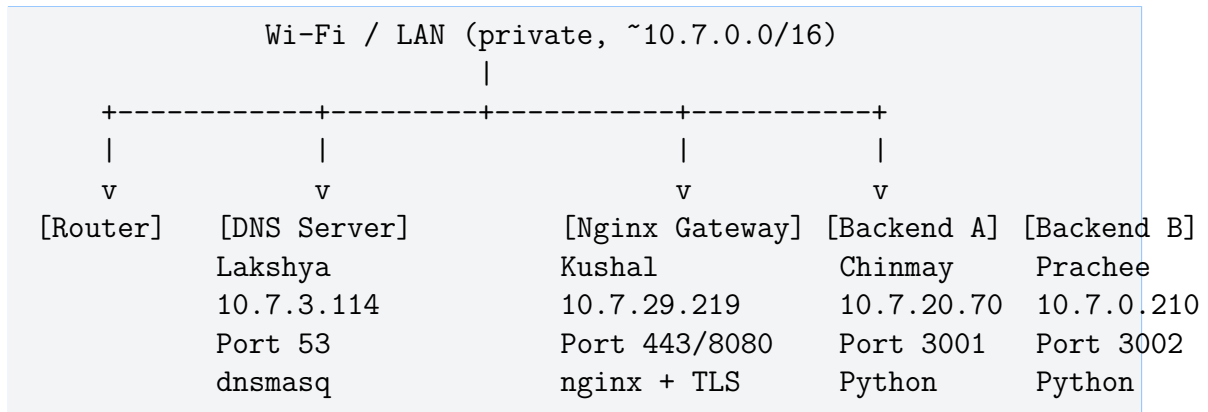


Figure 1: Physical network topology

Table 2: Local components mapped to cloud equivalents

Local Component	AWS Equivalent	GCP Equivalent
dnsmasq on Mac 1	Route 53 Private Hosted Zone	Cloud DNS
Nginx on Mac 2	Application Load Balancer	HTTP(S) Load Balancer
Backend A on Mac 3	EC2 Instance	Compute Engine VM
Backend B on Mac 4	EC2 Instance	Compute Engine VM
Private LAN	VPC	VPC Network
Self-signed CA	AWS Certificate Manager	GCP Certificate Manager

3.1 Overview

All four team machines were connected to the same private Wi-Fi network. Connectivity was verified by direct HTTP access from the gateway machine (Kushal, Mac 2) to both backend machines (Chinmay, Prachee).

3.2 Evidence — Direct Backend Access

```

=== Backend A ===
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.14.2
Content-Type: text/plain
Content-Length: 21
Cache-Control: max-age=60
X-Backend: A

Backend A is healthy

=== Backend B ===
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.14.7
Content-Type: text/plain
Content-Length: 21
Cache-Control: max-age=60
X-Backend: B

Backend B is healthy

```

Direct HTTP connectivity from Mac 2 to both Mac 3 (10.7.20.70:3001) and Mac 4 (10.7.0.210:3002) confirmed. Both backends returned 200 OK with the correct X-Backend identifier.

4 Task B — Private DNS Server

4.1 Configuration

dnsmasq was installed on Lakshya's machine (Mac 1, 10.7.3.114) and configured to resolve the .NAN.test zone while forwarding all other queries to public resolvers (8.8.8.8, 1.1.1.1), preserving normal internet connectivity.

Listing 1: dns/dnsmasq.conf

```
port=53
listen-address=10.7.3.114
bind-interfaces

# NAN project DNS records
address=/app.NAN.test/10.7.29.219
address=/api.NAN.test/10.7.29.219

# Forward external DNS to public resolvers
server=8.8.8.8
server=1.1.1.1

log-queries
log-facility=--
```

4.2 Evidence — DNS Resolution

Listing 2: dig output for app.NAN.test

```
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 6248

;; QUESTION SECTION:
;app.NAN.test.      IN  A

;; ANSWER SECTION:
app.NAN.test. 0    IN  A    10.7.29.219

;; SERVER: 10.7.3.114#53(10.7.3.114)
;; MSG SIZE rcvd: 57
```

Listing 3: dig output for api.NAN.test

```
;; ANSWER SECTION:
api.NAN.test. 0    IN  A    10.7.29.219
```

```
;; SERVER: 10.7.3.114#53(10.7.3.114)
```

Both `app.NAN.test` and `api.NAN.test` resolve to 10.7.29.219 (the Nginx gateway) using the team's private DNS server at 10.7.3.114:53. Status: NOERROR. Response is authoritative (aa flag set).

4.3 DNS vs. IP Layer Distinction

A key conceptual point demonstrated: DNS resolution and TCP/IP connectivity are independent layers.

Table 3: Layer independence demonstrated through DNS failure scenarios

Condition	DNS Result	TCP/IP Result
Correct DNS configured	Resolves correctly	Connection succeeds
Wrong DNS configured	Fails to resolve	Direct IP still reachable
DNS record wrong IP	Resolves (wrong IP)	Connection to wrong host
DNS server offline	Name lookup fails	Direct IP still reachable

5 Task C — Backend Services

5.1 Design

Two minimal Python HTTP servers were built. Both bind to 0.0.0.0 so they are reachable from all machines on the LAN. Each exposes two endpoints and identifies itself in every response via the `X-Backend` header.

Table 4: Backend API endpoints

Endpoint	Response Body	Headers
GET /	Backend A / Backend B	X-Backend: A or B
GET /api/status	Backend A is healthy	X-Backend: A, Cache-Control: max-age=60

5.2 Source Code — Backend A

Listing 4: backends/backend-a/server.py

```
from http.server import BaseHTTPRequestHandler, HTTPServer

HOST = "0.0.0.0"
PORT = 3001
BACKEND = "A"
```

```
class Handler(BaseHTTPRequestHandler):

    def do_GET(self):
        if self.path == "/api/status":
            body = f"Backend {BACKEND} is healthy\n".encode()
            self.send_response(200)
            self.send_header("Content-Type", "text/plain")
            self.send_header("Content-Length", str(len(body)))
            self.send_header("Cache-Control", "max-age=60")
            self.send_header("X-Backend", BACKEND)
            self.end_headers()
            self.wfile.write(body)
        else:
            body = f"Backend {BACKEND}\n".encode()
            self.send_response(200)
            self.send_header("Content-Type", "text/plain")
            self.send_header("Content-Length", str(len(body)))
            self.send_header("X-Backend", BACKEND)
            self.end_headers()
            self.wfile.write(body)

server = HTTPServer((HOST, PORT), Handler)
print(f"Backend {BACKEND} listening on {HOST}:{PORT}")
server.serve_forever()
```

Backend B is identical with `BACKEND = "B"` and `PORT = 3002`.

6 Task D — Nginx Reverse Proxy and Load Balancer

6.1 Configuration

Nginx on Mac 2 (10.7.29.219) acts as the single entry point. Clients never connect directly to backend IPs. Round-robin load balancing distributes requests across both backends.

Listing 5: nginx/nan.conf (abridged)

```
upstream nan_backends {
    server 10.7.20.70:3001;    # Backend A - Chinmay
    server 10.7.0.210:3002;   # Backend B - Prachee
}

server {
    listen 443 ssl;
    server_name app.NAN.test api.NAN.test;

    ssl_certificate      /opt/homebrew/etc/nginx/nan-server.crt;
    ssl_certificate_key  /opt/homebrew/etc/nginx/nan-server.key;

    ssl_protocols        TLSv1.2 TLSv1.3;
```

```

ssl_ciphers          HIGH:!aNULL:!MD5;

location / {
    proxy_pass http://nan_backends;
    proxy_set_header Host          $host;
    proxy_set_header X-Real-IP     $remote_addr;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_next_upstream error timeout http_502;
}
}

```

6.2 Traffic Path

```

Client
|
| HTTPS  app.NAN.test:443
v
Nginx Gateway  10.7.29.219:443
|
| Round-Robin  (plain HTTP on LAN)
|
+----[odd]-----> Backend A  10.7.20.70:3001
|
+----[even]-----> Backend B  10.7.0.210:3002

```

Figure 2: Request routing through Nginx to backends

6.3 Evidence — Load Balancing

Twelve consecutive HTTPS requests to `app.NAN.test` produced the following pattern:

Request 1: X-Backend: B	Request 7: X-Backend: B
Request 2: X-Backend: A	Request 8: X-Backend: A
Request 3: X-Backend: B	Request 9: X-Backend: B
Request 4: X-Backend: A	Request 10: X-Backend: A
Request 5: X-Backend: B	Request 11: X-Backend: B
Request 6: X-Backend: A	Request 12: X-Backend: A

Perfect round-robin distribution: Backend B served odd requests, Backend A served even requests, alternating across all 12 requests without interruption.

7 Task E — HTTPS and TLS

7.1 Certificate Authority Setup

A local private Certificate Authority named **NAN Local CA** was created using OpenSSL. The CA certificate was distributed to all client machines and added to the macOS system trust store, enabling certificate verification without the `-k` flag.

Listing 6: Adding NAN Local CA to macOS trust store

```
sudo security add-trusted-cert -d -r trustRoot \
-k /Library/Keychains/System.keychain nan-ca.crt
```

7.2 Server Certificate

Table 5: Server certificate properties

Property	Value
Subject (CN)	app.NAN.test
Issuer	NAN Local CA
Organisation	NAN
Country / State / City	IN / West Bengal / Kolkata
Valid From	Oct 4, 2026
Valid Until	Oct 4, 2027
TLS Version	TLSv1.3
Cipher Suite	AEAD-CHACHA20-POLY1305-SHA256

7.3 TLS Termination Architecture

TLS is terminated at Nginx. Backend communication uses plain HTTP on the trusted private LAN.

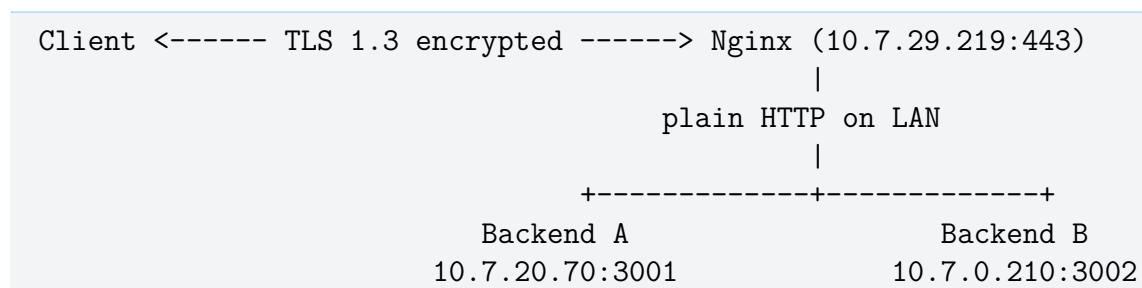


Figure 3: TLS termination at Nginx edge

7.4 TLS Handshake Observed

The `curl -v` output captured the complete TLS 1.3 handshake:

```
* (OUT), TLS handshake, Client hello (1)
* (IN), TLS handshake, Server hello (2)
* (IN), TLS handshake, Unknown (8)          <- TLS 1.3 extensions
* (IN), TLS handshake, Certificate (11)
* (IN), TLS handshake, CERT verify (15)
```

```
* (IN), TLS handshake, Finished (20)
* (OUT), TLS handshake, Finished (20)
* SSL connection using TLSv1.3 / AEAD-CHACHA20-POLY1305-SHA256
* SSL certificate verify ok.
```

Table 6: TLS 1.3 handshake message sequence

Message	Direction	Purpose
ClientHello	Client → Server	Advertises TLS versions, ciphers, SNI
ServerHello	Server → Client	Selects TLS 1.3, CHACHA20 cipher
Certificate	Server → Client	Sends <code>nan-server.crt</code>
CertVerify	Server → Client	Proves possession of private key
Finished	Both directions	Confirms handshake; session keys active

After *Finished*, all application data is encrypted. HTTP payload is not visible in Wireshark — only opaque TLS Application Data records appear.

HTTPS established without certificate warnings. SSL certificate verify ok. confirmed by curl. TLS 1.3 with CHACHA20-POLY1305 cipher in use.

8 Task F — HTTP Caching

8.1 Implementation

The `/api/status` endpoint includes a `Cache-Control: max-age=60` header in every response. This instructs clients and intermediate caches to treat the response as fresh for 60 seconds.

8.2 Evidence

Listing 7: `curl -sI https://api.NAN.test/api/status`

```
HTTP/1.1 200 OK
Server: nginx/1.31.6
Content-Type: text/plain
Content-Length: 21
Connection: keep-alive
Cache-Control: max-age=60
X-Backend: B
```

8.3 Cache Behaviour Explained

Table 7: HTTP cache states

State	Behaviour	HTTP Status
Fresh hit	Response served from local cache; no network request made	(none)
Conditional request	Client re-validates with If-None-Match or If-Modified-Since	304 Not Modified
Full new request	Cache expired or flushed; complete round-trip to backend	200 OK

9 Task G — Complete Protocol Flow Capture

9.1 OSI Layer Mapping

Every layer of the OSI model is exercised in a single client request:

Table 8: OSI layer mapping for the NAN request

Layer	Name	Protocol	Where / What
7	Application	DNS, HTTP/1.1	dnsmasq, Python backends, nginx
6	Presentation	TLS 1.3	Nginx: encrypts/decrypts
5	Session	TLS session	Key exchange, cipher negotiation
4	Transport	TCP (443, 3001, 3002) / UDP (53)	All machines
3	Network	IPv4 (10.7.x.x)	Wi-Fi LAN routing
2	Data Link	802.11 Wi-Fi frames	NIC on each machine
1	Physical	Wi-Fi radio signal	Access point

9.2 Port Identification

Table 9: Ports and socket pairs in a single request

Traffic	Protocol	Src Port	Dst Port	Machine
DNS query	UDP	Ephemeral	53	Client → Mac 1
DNS response	UDP	53	Ephemeral	Mac 1 → Client
HTTPS	TCP	Ephemeral	443	Client → Mac 2
Proxy to Backend A	TCP	Ephemeral	3001	Mac 2 → Mac 3
Proxy to Backend B	TCP	Ephemeral	3002	Mac 2 → Mac 4

Table 10: Wireshark capture files

File	Contents
NAN-phase1-dns-capture.pcapng	DNS query/response for <code>app.NAN.test</code>
NAN_phase1_baseline_traffic.pcapng	Full system: DNS + TCP + TLS + HTTP
NAN_phase1_backend_failure_failover.pcapng	Backend A down, failover to B

9.3 Wireshark Captures

Key Wireshark filters used during analysis:

```
dns                # DNS query and response packets
tcp.flags.syn == 1 # TCP three-way handshake
tls.handshake      # TLS ClientHello, ServerHello,
    Certificate
tcp.port == 443    # All HTTPS traffic
tcp.flags.reset == 1 # TCP RST (connection refused)
ip.addr == 10.7.3.114 # DNS server traffic
```

9.4 TCP Three-Way Handshake

Observed in the Wireshark baseline capture (`tcp.flags.syn == 1 && tcp.port == 443`):

```
Client      --> [SYN]      seq=X          -->
    10.7.29.219:443
10.7.29.219 --> [SYN-ACK]  seq=Y, ack=X+1  --> Client
Client      --> [ACK]      ack=Y+1        -->
    10.7.29.219:443
```

The handshake completes before any TLS or application data is transmitted, confirming TCP’s reliable, connection-oriented nature.

10 Phase 1 Failure Demonstrations

10.1 Backend Failover — Detailed

When Backend A (Mac 3) was stopped:

```
Before failure:  X-Backend: A, B, A, B, A, B ...
After Backend A stopped: X-Backend: B, B, B, B, B ...
After Backend A restarted: X-Backend: A, B, A, B ...
```

Nginx’s passive health check (`proxy_next_upstream error timeout`) detected the TCP connection failure to `10.7.20.70:3001` and automatically routed subsequent requests to `10.7.0.210:3002` (Backend B). The client received no errors during the failover.

Table 11: Phase 1 mandatory failure scenarios

Failure Injected	Observed Behaviour	Layer Affected
Wrong DNS resolver on client	Name resolution fails; direct IP still reachable	Application (DNS, L7)
DNS record points to wrong IP	DNS succeeds; TCP connection reaches wrong host	Network (L3)
One backend stopped	Nginx routes all traffic to remaining backend	Application upstream
Both backends stopped	Nginx returns 502 Bad Gateway	Application upstream
Wrong destination port	TCP Connection refused; host reachable	Transport (TCP, L4)

10.2 502 Bad Gateway

When both backends were stopped simultaneously, Nginx returned:

```
HTTP/1.1 502 Bad Gateway
Server: nginx/1.31.6
```

This confirms the boundary between the edge proxy layer and the application backend layer: the TLS handshake still succeeds (Nginx is reachable), but the upstream HTTP connection cannot be established.

11 Security Notes

The following files contain private cryptographic material and are **not committed** to the repository. They reside only on the gateway machine:

- `nan-server.key` — Nginx TLS private key
- `nan-ca.key` — NAN Local CA private key

A `.gitignore` rule (`*.key, *.pem`) prevents accidental commit.

Only the public server certificate (`tls/nan-server.crt`) and public CA certificate are distributed and committed.

12 Project File Structure

```
cn-phase1/
```

```

|-- README.md
|-- architecture/
|   |-- topology.md           Network diagram, IP table, cloud
|   |   equivalents
|   |-- request-flow.md       Step-by-step protocol flow, OSI
|   |   mapping
|-- backends/
|   |-- backend-a/
|   |   |-- server.py         Backend A (port 3001)
|   |-- backend-b/
|   |   |-- server.py         Backend B (port 3002)
|-- dns/
|   |-- dnsmasq.conf          dnsmasq config with .NAN.test
|   |   records
|-- nginx/
|   |-- nan.conf              Reverse proxy, load balancer, TLS
|   |   config
|-- tls/
|   |-- README.md             TLS setup, CA details, trust store
|   |-- nan-server.crt        Public server certificate
|-- scripts/
|   |-- verify.sh             Pre-demo automated checks
|   |-- failure-demo.sh       Failure scenario commands
|-- docs/
|   |-- submission-notes.md    Task completion status, checklist
|-- evidence/
|   |-- 01-lan/               LAN connectivity proof
|   |-- 02-dns/               DNS resolution output
|   |-- 03-https/            TLS handshake verification
|   |-- 04-load-balancing/    Round-robin load balancing log
|   |-- 05-cache/            Cache-Control header demonstration
|   |-- 06-wireshark/        Wireshark captures and screenshots
|   |-- 07-failure/          Backend failure and failover
|   |   evidence

```

13 Summary

Table 12: Phase 1 task completion summary

Task	Description	Status	Evidence
A	Establish Private LAN	Complete	evidence/01-lan/
B	Configure Private DNS	Complete	evidence/02-dns/
C	Build Two Backend Services	Complete	backends/
D	Nginx Reverse Proxy + Load Balancer	Complete	evidence/04-load-balancing/
E	HTTPS / TLS	Complete	evidence/03-https/
F	HTTP Caching	Complete	evidence/05-cache/
G	Complete Protocol Flow Capture	Complete	evidence/06-wireshark/
Failure Scenarios		Complete	evidence/07-failure/

All Phase 1 mandatory tasks are complete. The NAN platform successfully demonstrates the full DNS $\rightarrow$ TCP $\rightarrow$ TLS $\rightarrow$ HTTP $\rightarrow$ Load Balancer $\rightarrow$ Backend stack on a private local network, with Wireshark evidence at every layer and controlled failure scenarios showing layer independence.